

SIMPLY FPU

by **Raymond Filiatreault**
Copyright 2003

Chap. 12 Other instructions

The instructions covered in this chapter are those which could not be included in other more specific categories. They are (in alphabetical order):

FNOP	No OPeration
FPREM	Partial REMainder
FPREM1	Partial REMainder 1
EXTRACT	eXTRACT exponent and significand

FNOP (No Operation)

Syntax: **fnop** (no operand)

Exception flags: None

This instruction performs no operation. It only takes up space and consumes a few clock cycles. It may be used as a filler for alignment purposes or timing delays.

FPREM (Partial remainder)

Syntax: **fprem** (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
Underflow

This instruction performs the equivalent of a ST(0) modulo ST(1) with REAL numbers, overwriting the content of ST(0) with the remainder which will have the same sign as the original value in ST(0). The content of ST(1) remains unchanged.

The remainder represents the following value:

$$\text{Remainder} = \text{ST}(0) - (Q * \text{ST}(1))$$

where Q is an integer obtained by truncating the quotient of $\text{ST}(0) / \frac{1}{2}\text{ST}(1)$.

However, the exponent in ST(0) cannot be reduced by more than 63 with each execution of the FPREM instruction. If the difference between the exponents of ST(0) and ST(1) is larger than 63, ST(0) will then be only partially reduced and that partial remainder will replace the content of ST(0). The software will have to repeat the FPREM instruction as often as necessary to obtain a final remainder which will be smaller than the modulus ST(1). When the reduction is complete, the C2 bit of the [Status Word](#) is cleared, otherwise it is set to 1.

An **Invalid operation** exception is detected if ST(0) or ST(1) is empty, or is a [NAN](#), or if ST(0) is [INFINITY](#), or if ST(1) is zero, setting the related flag in the Status Word. The content of ST(0) would be overwritten with the [INDEFINITE](#) value.

A **Stack Fault** exception is also detected if either ST(0) or ST(1) is empty, setting the related flag in the Status Word.

A **Denormal** exception is detected when the content of either ST(0) or ST(1) is a [denormalized](#) number or a result is a denormalized number, setting the related flag in the Status Word.

An **Underflow** exception will be detected if the result exceeds the range limit of [REAL10](#) numbers, setting the related flag in the Status Word.

When the operation is successful, the 3 least significant bits of the quotient (Q) are inserted in the C0, C1 and C3 bit fields of the Status Word in the following manner:

C0 = bit2 of the quotient (Q2)

C1 = bit0 of the quotient (Q0)

C3 = bit1 of the quotient (Q1)

If necessary, the following code is suggested to unscramble the bits of the Status Word to their original order in the quotient.

```

;               AH                      AL
fstsw ax        ;x  Q1 x  x  x C2 Q0 Q2          x  x  x  x  x  x  x  x
fwait
sahf            ;copy AH to the CPU flag register
jpe  more_fprem ;C2 became the parity flag PF
;if set, the FPREM instruction needs to be repeated
;Q2 became the carry flag CF
rcl  al,1       ;x  Q1 x  x  x  x  Q0 Q2          x  x  x  x  x  x  x  Q2
shl  ah,2       ;x  x  x  x  Q0 Q2 x  x  (Q1 -> CF) x  x  x  x  x  x  x  Q2
rcl  al,1       ;x  x  x  x  Q0 Q2 x  x          x  x  x  x  x  x  x  Q2 Q1
shr  ah,4       ;x  x  x  x  x  x  x  x  (Q0 -> CF) x  x  x  x  x  x  x  Q2 Q1
rcl  al,1       ;x  x  x  x  x  x  x  x          x  x  x  x  x  x  Q2 Q1 Q0
and  eax,7      ;keep only the 3 least significant bits

```

The FPREM instruction was almost an absolute necessity with the earlier FPUs (prior to the 387) when the source angle for the FPTAN instruction could only be in the range of 0 to $\pi/4$ radian. The lower 3 bits of the quotient Q were also necessary to locate the angle in the proper sector in order to determine if the reciprocal of the result needed to be computed and which sign was required.

With the more modern FPUs (387+), the FPREM instruction may still be required before using one of the [trigonometric instructions](#) if there is a risk that the source angle exceeds the allowable range of $i_6 1/2 2^{63}$ radians. However, knowing the lower 3 bits of the quotient has become almost useless. It is still being made available primarily for backward compatibility.

The following code is suggested for reducing an angle within the allowable range of $i_6 1/2 2^{63}$ radians. It assumes that the angle value is already in ST(0) and already expressed in radians.

```

; ST(0)=angle in radians to be reduced
fldpi          ;->ST(0)= $\pi$ , ST(1)=angle
fadd  st,st    ;->ST(0)= $2\pi$ , ST(1)=angle
fxch          ;->ST(0)=angle, ST(1)= $2\pi$ 
@@:
fprem          ;->ST(0)=reduced angle, ST(1)= $2\pi$ 
fstsw  ax      ;Status Word into AX
fwait         ;insure transfer is completed
sahf          ;copy to the CPU flags
jpe  @B        ;continue reducing if C2=PF=1 (reduction incomplete)

fstp  st(1)    ;overwrite the  $2\pi$  with the ST(0) value and POP ST(0)
; ->ST(0)=reduced angle smaller than  $2\pi$ 

```

FPREM1 (Partial remainder 1)

Syntax: **fprem1** (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value, Underflow

This instruction is a modification of the FPREM instruction to compute the remainder as specified in the IEEE Std 754. According to that Standard, the magnitude of the remainder must be less than or equal to half of the modulus ST(1). The sign of the remainder can thus be different than the sign of the original value in ST(0).

For example, using integers, 11 modulo 7 with the FPREM instruction would be +4. With the FPREM1 instruction, 4 being more than half the modulus 7, the resulting remainder would be -3. (10 modulo 7 would be +3 with either instruction.)

(See the [FPREM](#) instruction for other details.)

FXTRACT (Extract exponent and significand)

Syntax: **fxtract** (no operand)

Exception flags: Stack Fault, Invalid operation, Denormalized value,
Zero divide

This instruction splits the content of ST(0) into two parts, the true **unbiased exponent** and the **significand**. The content of ST(0) is overwritten with the true unbiased exponent in floating point format, the TOP field of the [Status Word](#) is decremented, and the significand is inserted in the new ST(0) with a biased exponent of 0 (3FFFh for the [REAL10](#) format of the data registers).

If ST(0) originally had a value of ± 0 , the true unbiased exponent would be set at [-INFINITY](#) and the significand would be set to a value of 0 with the same sign as the original one.

If ST(0) originally had a value of $\pm \text{INFINITY}$, the true unbiased exponent would be set at $+\text{INFINITY}$ and the significand would be set at INFINITY with the same sign as the original one.

An Invalid operation exception is detected if ST(0) is empty, or is a [NAN](#), setting the related flag in the Status Word. The content of ST(0) would be overwritten with the [INDEFINITE](#) value, the TOP field of the Status Word decremented, and the INDEFINITE value inserted in the new ST(0).

A Stack Fault exception is also detected if ST(0) is empty, setting the related flag in the Status Word.

A Denormal exception is detected when the content of ST(0) is a [denormalized](#) number, setting the related flag in the Status Word.

A Zero divide exception is detected when the content of ST(0) is 0.

This instruction could be used for logarithmic scaling operations. The extracted exponent would be multiplied by the scaling factor, its antilog computed with the [F2XM1](#) instruction, and that result multiplied by the extracted significand.

[RETURN TO](#)
[SIMPLY FPU](#)
[CONTENTS](#)